

MODULE ALGORITHMIQUE ET STRUCTURES DE DONNÉES AVANCÉES

ALGORITHMES DE TRI (SUITE)

©B. DERDOURI

TriFusion

Module AlgoIII- L3I-S5-U-CERGY

9/17/2023

1

PLAN DE COURS

- **Approche Diviser pour Mieux Régner**
- **Tri Fusion**
 - Principe, Exemple, Algorithme
- **Fusion de deux tableaux triés**
 - Principe
 - Exemple
 - Version 1 : algorithme fusion de deux tableaux triés
 - Version 2 : fusion avec sentinelle
- **Version 1 : Algorithme fusionner**
 - Principe, Exemple, Algorithme
- **Version 2 : Algorithme fusionner**
 - Principe, Exemple, Algorithme

©B. DERDOURI

TriFusion

Module AlgoIII- L3I-S5-U-CERGY

9/17/2023

2

APPROCHE DIVISER POUR MIEUX RÉGNER

- **Principe :**

- L'approche diviser pour mieux régner est basée sur **le principe de division** : il divise un problème en plusieurs sous problèmes similaires au problème initial, mais de taille inférieure, résolvent les sous problèmes de façon récursive, puis combinent ses solutions pour retrouver une solution au problème initial.
- L'approche 'Diviser pour Mieux Régner' donne lieu à trois étapes à chaque niveau de récursivité :
- **Diviser** le problème en un certain nombre de sous problèmes
- **Régner** sur les sous problèmes en les résolvant récursivement. Par ailleurs, si la taille d'un sous problème est assez réduite, on peut le résoudre directement
- **Combiner** les solutions des sous problèmes en une solution complète pour le problème initial.

TRI FUSION

- **Principe :**

- L'algorithme de tri par fusion suit très fidèlement la règle de 'Diviser pour Mieux Régner', intuitivement, il agit de la manière suivante :
- **Diviser** : Diviser les éléments du tableau à trier en deux sous tableaux de $n/2$ éléments
- **Régner** : Trier les deux sous tableaux récursivement à l'aide du tri par fusion
- **Combiner** : Fusionner les deux sous tableaux triés pour produire le tableau initial trié.
- **Remarque**
- La récursivité prend fin quand le sous tableau à trié est soit vide soit à une longueur de 1, auquel cas il n'y a plus rien à faire, puisque toute séquence vide ou de longueur 1 est déjà trié.

ALGORITHME DE TRI PAR FUSION

Procédure triFusion(indinf, indsup, tab)

P.F : indinf, indsup : Entier (E)

tab : tableau (E/S)

Début

Si indinf < indsup **Alors**

Indmil $\leftarrow$ (indinf+indsup) / 2

triFusion(indinf, indmil, tab)

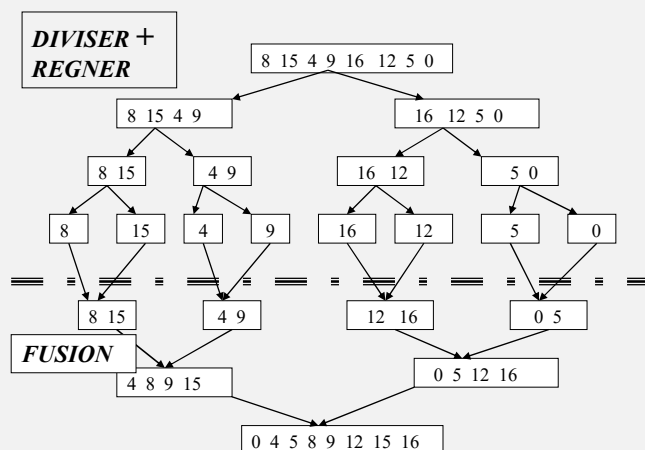
triFusion(indmil+1, indsup, tab)

Fusionner(indinf, indmil, indsup, tab)

Finsi

Fin

EXEMPLE



FUSION DE DEUX TABLEAUX TRIÉS

- **Problème :**
- Ce problème consiste à construire à partir de plusieurs tableaux triés, un seul tableau trié suivant un certain critère de fusion.
- **Hypothèse**
- Soient deux tableaux triés $\text{tab1}[1..n1]$ et $\text{tab2}[1..n2]$ à valeurs dans un même ensemble E et munis de la même relation d'ordre $\leq$. On souhaite construire un tableau $\text{tab3}[1..n3]$ trié et contenant tous les éléments de tab1 et tab2 ($n3 = n1 + n2$).
- **Méthode 1**
- * Parcourir simultanément les deux tableaux tab1 et tab2 et prendre à chaque fois le plus petit élément.
- * Des qu'un tableau est terminé, on complète le tableau résultant par tous les éléments restants dans le tableau non terminé.

©B. DERDOURI

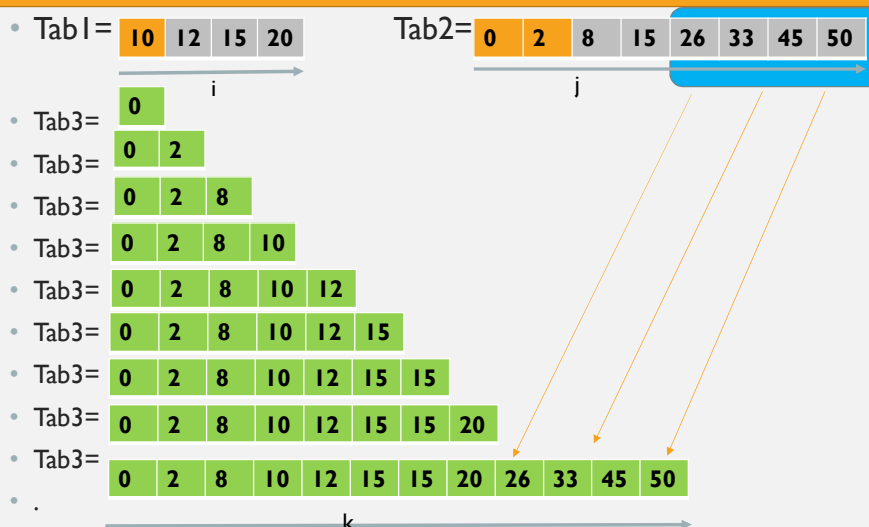
TriFusion

Module Algorithme L3I-S5-U-CERGY

9/17/2023

7

EXEMPLE



©B. DERDOURI

TriFusion

Module Algorithme L3I-S5-U-CERGY

9/17/2023

8

ALGORITHME DE FUSION DE DEUX TABLEAUX TRIÉS

Procédure fusionDeuxTableauxTriés(n1, n2, n3, tab1, tab2, tab)

P.F : n1, n2 : Entier (E) n3: Entier (S)
 tab1, tab2 : tableau (E) tab3 : tableau (S)

Début

```

i ← j ← k ← 1
n3 ← n1 + n2
Tantque (i ≤ n1) et (j ≤ n2) Faire
    Si (tab1[i] ≤ tab2[j]) Alors
        tab[k] ← tab1[i]
        i ← i + 1
    Sinon
        tab[k] ← tab2[j]
        j ← j + 1
    Finsi
    k ← k + 1
Fintantque
  
```

VERSION 1 : ALGORITHME DE FUSION DE DEUX TABLEAUX TRIÉS

- //i > n1 ou j > n2 : une seule, au plus des 2 boucles ci dessus sera exécutée

```

tantque i ≤ n1 faire
    tab[k] ← tab1[i]
    i ← i + 1
    k ← k + 1
Fintantque
  
```

```

//pour l variant i à n1 faire
//    tab[k] ← tab1[l]
//    k ← k + 1
//Finpour
  
```

```

tantque j ≤ n2 faire
    tab[k] ← tab2[j]
    j ← j + 1
    k ← k + 1
Fintantque
  
```

```

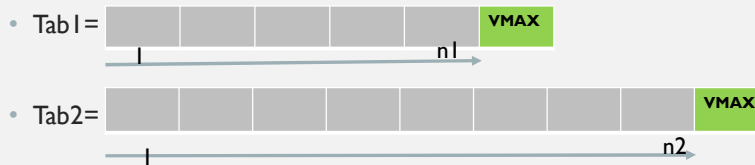
//pour l variant j à n2 faire
//    tab[k] ← tab2[l]
//    k ← k + 1
//Finpour
  
```

FIN

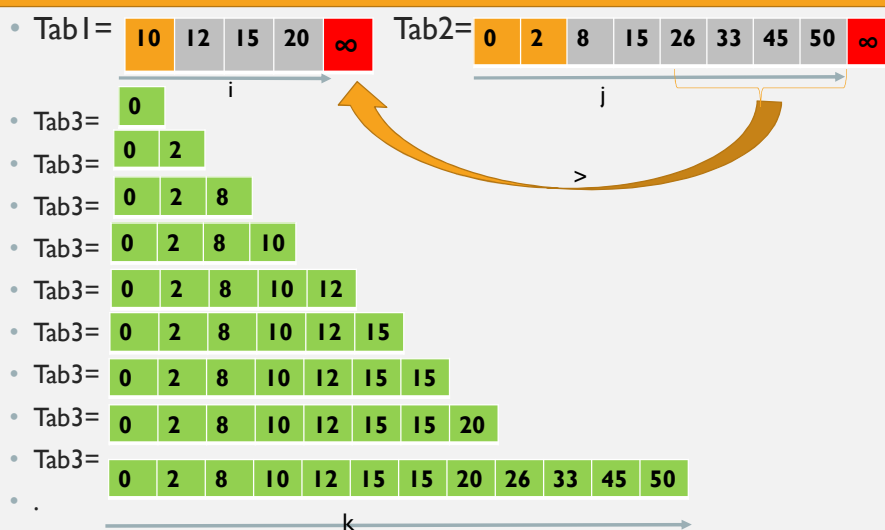
MÉTHODE 2 : FUSION AVEC SENTINELLE

• Principe :

- Cet algorithme utilise une sentinelle 'VMAX' supérieure à tous les éléments des deux tableaux tab1 et tab2 et destinée à copier dans le tableau résultant tab, les éléments restant dans l'un des tableaux tab1 ou tab2 après que l'autre ait vidé.



EXEMPLE



VERSION I : ALGORITHME FUSIONNER

```

procédure Fusionner(indinf, indmil, indsup, tab)
P.F : indinf, indmil, indsup : Entier (E)
      tab : tableau (E/S)
Début
    //Construction du tableau tab1
    pour i variant de indinf à indmil faire
        tab1[i-indinf+1] ← tab[i]
    finpour
    //Construction du tableau tab2
    pour i variant de indmil+1 à indsup faire
        tab2[i-indmil] ← tab[i]
    finpour
    n1 ← indmil – indinf + 1
    n2 ← indsup – indmil
    // Fusion de tab1 et tab2 => tab=tab1+tab2

```

VERSION I : ALGORITHME FUSIONNER

```

i ← 1 // Parcours du tab1
j ← 1 // Parcours du tab2
k ← indInf // Parcours du tab
Tantque (i <= n1) et (j <= n2) Faire
    Si (tab1[i] <= tab2[j]) Alors
        tab[k] ← tab1[i]
        i ← i + 1
    Sinon
        tab[k] ← tab2[j]
        j ← j + 1
    Finsi
    k ← k + 1
Fintantque

```


VERSION 2 : ALGORITHME FUSIONNER

```

procédure Fusionner(indinf, indmil, indsup, tab)
P.F : indinf, indmil, indsup : Entier (E)
      tab : tableau (E/S)
Début
    //Construction du tableau tab1
    pour i variant de indinf à indmil faire
        tab1[i-indinf+1] ← tab[i]
    finpour
    //Construction du tableau tab2
    pour i variant de indmil+1 à indsup faire
        tab2[i-indmil] ← tab[i]
    finpour
    // Ajout d'une sentinelle
    tab1[indmil - indinf + 2] ← VMAX
    tab2[indsup - indmil + 1] ← VMAX
    // Fusion de tab1 et tab2 => tab=tab1+tab2

```

VERSION 2 : ALGORITHME FUSIONNER

```

i ← 1      // Parcours du tab1
j ← 1      // Parcours du tab2

pour k variant de indInf à indSup Faire
    Si (tab1[i] <= tab2[j])Alors
        tab[k] ← tab1[i]
        i ← i + 1
    Sinon
        tab[k] ← tab2[j]
        j ← j + 1
    Finsi
Fintantque
FIN

```